Fusion ERP Lab — Assignment #6 & #6B

A Pilot Study & Engineering Report: Implementing and Integrating the Employee Leave Management (ELM) Module Using Local Code-LLMs and Human-AI Pair Engineering

Student Name: Mohit Jain | **Roll Number:** 24BCS151 | **Email:** 24bcs151@iiitdmj.ac.in

PDPM Indian Institute of Information Technology, Design and Manufacturing, Jabalpur

Date: October 2026 | **Target Repository:** FusionIIIT/Fusion-Integrated

1 Executive Summary

This report documents the two-phase experimental study conducted for Fusion ERP Lab Assignments #6 and #6B. In **Phase 1**, two locally deployable code language models—**Qwen 2.5 Coder 7B** and **DeepSeek Coder 6.7B**—were evaluated under identical task instructions to implement the Employee Leave Management (ELM) module inside the unified Fusion Base Code. In **Phase 2**, the Qwen 2.5 Coder implementation was selected as the superior starting point and progressively reviewed, tested, corrected, and integrated into a reliable, deployment-ready state adhering to Fusion ERP's multi-database architecture and Mantine UI standards.

All platform verification gates, including `make check` (Ruff, import-linter, ops checks, schema checks, and 387 pytests) and `make check-client` (TypeScript strict check, Vitest suite, and Vite production bundle), pass with zero errors.

2 Baseline Architecture & Integration Context

Fusion ERP enforces strict architectural boundaries to support a scalable, modular institute management system:

- **Multi-Database Identity Separation:** Authentication and user identity live in an independent IAM service and database (`fusion_system_db`). Module business tables live in `fusion_integrated`. Therefore, direct Django foreign keys to `User` or `Principal` are illegal; all person references must use plain integer `user_id` via `core.db.mixins.UserScopedModel`.
- **Server-Driven RBAC Navigation:** The frontend draws no hardcoded navigation. Routes and sidebar items are rendered dynamically based on permissions published in `registry.py` and seeded into the IAM service via `seed_iam_permissions`.
- **Layering & Contracts:** Logic must be decoupled into domain services (`services.py`) for state mutations, selectors (`selectors.py`) for scoped reads, and plural contracts (`contracts.py`). Cross-module imports are enforced by CI through `.importlinter`.
- **Frontend Design System:** The client uses Mantine UI (`@mantine/core`, `@tabler/icons-react`) and TanStack React Query hooks connected via a centralized HTTP client carrying session cookies and CSRF headers.

3 Phase 1: Local Code-LLM Pilot Evaluation

3.1 Experimental Setup

- **Operating System:** Linux x86_64 (Ubuntu-based)
- **Inference Runtime:** Ollama (v0.86.2 runtime via Aider CLI tool)
- **Context Configuration:** 2048 repo-map tokens, full ELM specification loaded read-only (`../ELM_FULL_SPECS.txt`), core settings and URL configurations.
- **Common Task Prompt:** Instructed the model to inspect the Fusion Base repository and ELM specs, reuse existing login, role management, common UI and backend infrastructure, and implement the necessary data models, services, APIs, and frontend pages.

3.2 Model 1 Evaluation: Qwen 2.5 Coder 7B (`ollama/qwen2.5-coder:7b`)

- **Observed Behavior:** The model successfully understood that both backend and frontend components were needed. It attempted a repository-wide generation, outputting definitions for models (`LeaveBalance`, `LeaveApplication`, `SubstituteRequest`, `HolidayCalendar`), views, serializers, URLs, and React components.
- **Limitations & Defects:**
  1. Context window saturation: Chat context reached 32,806 tokens, exceeding the 32,768 limit.
  2. Patch format failure: The model generated markdown code blocks rather than Aider's diff blocks, requiring manual extraction into commit `fce52c6`.
  3. Architectural mismatches: Used direct `ForeignKey(Principal)` relations; used unstyled HTML `<table>` elements instead of Mantine components; lacked `registry.py` RBAC declarations.
- **Initial Build/Run Status:** Code generated and integrated, but does not run successfully.

3.3 Model 2 Evaluation: DeepSeek Coder 6.7B (`ollama/deepseek-coder:6.7b`)

- **Observed Behavior:** DeepSeek Coder explicitly refused repository-level generation, stating: *"As per your request, I can't provide a full implementation here due to complexity and size of the project."* When asked about `manage.py`, it hallucinated that the file was unrelated to software development.
- **Fallback:** Under a subsequent prompt, it produced a minimal Django snippet in chat relying on `django.contrib.auth.models.User`. It produced zero frontend code and no service layer.
- **Initial Build/Run Status:** Could not obtain a usable integrated implementation.

3.4 Selection Rationale for Phase 2

Qwen 2.5 Coder 7B was selected because it made a genuine, multi-file full-stack implementation attempt. It provided a structured foundation that could be systematically refactored, whereas DeepSeek Coder required complete manual construction from scratch.

4 Phase 2: Systematic Review, Testing, and Improvement

4.1 Step 1 & 2: Understand and Build/Run Engineering

Initial testing identified that running the system locally required three concurrent processes: IAM backend (port 8001), Fusion Base backend (port 8002), and Vite frontend (port 5173).

- **Environment Collision:** `DJANGO_SETTINGS_MODULE` from Fusion Base leaked into the IAM service, crashing it with `ModuleNotFoundError: No module named 'config'`.
- **HTTPS Redirection:** Host shell variable `DEBUG=release` caused Django to enforce `SECURE_SSL_REDIRECT` causing port 8001 connection refused errors.
- **Solution:** Created a unified orchestrator script (`run.sh` / `make run`) that isolates subshell environments, forces `DEBUG=True`, manages background process traps, and includes HTTP readiness polling.

4.2 Step 3 & 4: Review and Problem Identification

A thorough code and specification review revealed 10 major defects:

1. **Illegal Cross-Database Foreign Keys:** `ForeignKey(Principal)` broke Postgres multi-database isolation.
2. **Missing RBAC & Navigation:** No `modules/elm/registry.py` existed; IAM permissions were never seeded.
3. **Boundary Contract Violations:** `modules.elm` was absent from `.importlinter`.

4. **Layering Degradation:** Business logic was coupled directly inside DRF viewsets.
5. **Frontend Incompatibility:** Generic HTML tables lacked Mantine styling and responsive layouts.
6. **Absence of Test Coverage:** Zero unit, integration, or client tests existed.
7. **Schema Gaps:** Missing DRF Spectacular `@extend_schema` decorators triggered OpenAPI build warnings.

4.3 Step 5 & 6: Code Improvement and Verification

Through systematic human developer implementation assisted by targeted prompt queries, the following commits were engineered on branch `feat/elm-phase2`:

- **Commit 1 (`882dcac`):** Converted models to `UserScopedModel` (indexed `user_id` integer), updated `.importlinter`, and generated clean migration `0001_initial.py`.
- **Commit 2 (`8a93790`):** Implemented `services.py` (leave application, balance checks, cancellations, substitute workflow), `selectors.py` (scoping), `registry.py`, and 7 pytest test cases.
- **Commit 3 (`76302ba`):** Rebuilt client using Mantine UI (`MyLeavesPage`, `LeaveBalancesPage`, `LeaveApprovalsPage`, `HolidayCalendarPage`), React Query hooks, and Vitest test suite.
- **Commit 4–7 (`c1d37cb`, `7f2406e`, `275bd42`, `49e2822`):** Unified runner script, resolved port 8001/8002 settings isolation, added faculty/staff role grants, and automated IAM permission seeding.
- **Commit 8 (`7c9a0fd`):** Documented Phase 2 interaction logs, issue logs, and verification evidence.

5 Verification Evidence & Quality Gates Summary

Quality Gate / Check	Scope & Command Executed	Result
Django System Check	`manage.py check`	**Passed** (0 issues)
Migrations Verification	`manage.py makemigrations --check --dry-run`	**Passed** (Clean)
Python Code Style	`ruff check .`	**Passed** (0 warnings)
Architecture Boundaries	`lint-imports` (4 contracts, 212 files analyzed)	**Passed** (4 kept, 0 broken)
Cross-Module Foreign Keys	`ops/checks/no_cross_module_fk.py`	**Passed** (Clean)
Contract Signatures	`ops/checks/contracts_are_plural.py`	**Passed** (Clean)
Route & Nav Alignment	`ops/checks/nav_matches_routes.py`	**Passed** (In sync)
Permission Manifest	`manage.py permission_manifest --check`	**Passed** (In sync)
OpenAPI Specification	`make schema-check`	**Passed** (Matches code)
Backend Test Suite	`pytest` (387 platform & ELM tests)	**Passed** (387 passed)
Client Type Safety	`tsc --noEmit`	**Passed** (0 errors)
Frontend Unit Tests	`vitest run` (37 client tests, 5 files)	**Passed** (37 passed)
Production Bundle Build	`vite build`	**Passed** (Clean build)

6 Key Learnings on Human-AI Software Engineering

1. **Context Limits in Local LLMs:** 7B parameter models running locally on standard developer hardware hit severe context window degradations (32k token limits) when fed full repositories and extensive specification documents.
2. **Prompt Granularity:** Broad, unconstrained prompts (*"Implement the whole ELM module"*) trigger hallucinations and boilerplate standard Django code. Prompting must be broken down layer-by-layer (models → services → permissions → UI).

3. **Irreplaceability of Human Architectural Judgment:** Local LLMs cannot infer non-standard architectural conventions (e.g., multi-database identity isolation, server-driven RBAC navigation) unless guided by explicit human oversight and reference implementations.

7 Submission Links & Artifacts

- **Phase 2 Final ELM Branch:** mohitjaint/Fusion-Integrated (Branch: feat/elm-phase2)
- **Phase 2 ELM Module Code:** modules/elm/
- **Phase 2 AI Interaction Log:** phase2_ai_interaction_log.md
- **Phase 2 Verification Evidence:** phase2_verification_evidence.md
- **Phase 2 Issue & Improvement Log:** phase2_issue_improvement_log.md
- **Pull Request to Main:** FusionIIIT/Fusion-Integrated (PR: Assignment 6B)